

RELATÓRIO — CONCORRÊNCIA E TRANSAÇÕES

Experimentos, discussão e implementação de uma procedure de reserva em MySQL/InnoDB

Resultados dos Experimentos

Experimento A — Sem FOR UPDATE (Condição de Corrida)

Duas sessões tentaram ler e reservar o mesmo assento simultaneamente. Como não houve bloqueio explícito, ambas leram o assento como **DISPONIVEL**. A Sessão 1 obteve sucesso. A Sessão 2, baseada na leitura desatualizada, tentou prosseguir e falhou graças à restrição estrutural do banco.

Erro retornado (Sessão 2): #1062 — Duplicate entry '1' for key 'reservas.uq_reserva_assento'.

Experimento B — Com FOR UPDATE (Bloqueio Pessimista)

A Sessão 1 utilizou **SELECT ... FOR UPDATE** no assento 1. Quando a Sessão 2 tentou executar o mesmo comando, ela ficou travada aguardando o fim da primeira transação. Após o COMMIT da Sessão 1, a Sessão 2 foi liberada, mas, ao realizar a leitura, percebeu que o assento não estava mais disponível, exigindo que a operação fosse cancelada com ROLLBACK.

Experimento C — Deadlock (Espera Circular)

A Sessão 1 travou o assento 1 e, logo após, a Sessão 2 travou o assento 2. Em seguida, a Sessão 1 tentou travar o assento 2, e a Sessão 2 tentou travar o assento 1. O motor do InnoDB detectou a espera circular e forçou o aborto de uma das transações.

Resultado: Erro 1213 — Deadlock found. O experimento evidencia a necessidade de adquirir bloqueios sempre na mesma ordem.

Respostas para Discussão

1.

Entre o momento da consulta e a atualização (escrita), outra transação pode ler o mesmo dado e tentar reservá-lo, gerando uma condição de corrida em que ambas acham que o assento está livre.

2.

Aplicar um lock de escrita sobre as linhas lidas, impedindo que outras transações atualizem ou também apliquem bloqueios nelas.

3.

Quando a transação é concluída, com COMMIT ou ROLLBACK.

4.

Fica em estado de *lock wait* (espera), aguardando a liberação do recurso ou o estouro do timeout.

5.

Durante a sua espera na fila, a primeira transação pode ter alterado o status do assento para **RESERVADO**.

6.

É uma falha de concorrência que ocorre quando o resultado final muda dependendo da temporização incontrolável de eventos concorrentes.

7.

O bloqueio (Locking) impede fisicamente o acesso simultâneo. O MVCC fornece “snapshots” antigos e consistentes do dado para leitura enquanto outra transação escreve, permitindo concorrência sem bloqueio de leitura.

8.

Ocorre em esperas circulares: a Transação A possui X e aguarda Y, enquanto a Transação B possui Y e aguarda X.

9.

Porque elimina o ciclo cruzado de esperas. Se todos bloquearem primeiro o recurso de menor ID, nenhuma transação pegará o maior antes do menor.

10.

READ COMMITTED: lê dados confirmados; cada consulta tira um snapshot novo.

REPEATABLE READ: padrão MySQL/InnoDB; a transação inteira utiliza o mesmo snapshot criado no início.

SERIALIZABLE: leituras ganham bloqueios explícitos automaticamente, forçando uma execução estritamente sequencial.

11.

Atua como a última linha de defesa estrutural, protegendo o banco contra falhas na lógica do código ou acessos diretos que ignorem validações.

12.

Capturando a exceção disparada pelo SGBD e executando um padrão de retentativa (Retry), aguardando um curto intervalo para realizar a operação novamente.

Desafio Prático — Procedure de Reserva

A Stored Procedure desenvolvida no MySQL trata a concorrência utilizando **FOR UPDATE** para serializar o acesso, transaciona as tabelas em conjunto para garantir o modelo ACID e implementa tratamento automático de erros.

```
DELIMITER $$
CREATE PROCEDURE ReservarAssento(
  IN p_passageiro_id BIGINT,
  IN p_voo_id BIGINT,
  IN p_numero_assento VARCHAR(10)
)
BEGIN
  DECLARE v_assento_id BIGINT;
  DECLARE v_status VARCHAR(30);

  -- Tratamento de erro (ROLLBACK automático)
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SELECT 'Erro técnico na transação. Rollback executado.' AS Resultado;
  END;
```

```

START TRANSACTION;

SELECT id, status INTO v_assento_id, v_status
FROM assentos
WHERE voo_id = p_voo_id AND numero = p_numero_assento
FOR UPDATE;

IF v_assento_id IS NULL THEN
ROLLBACK;
SELECT 'Assento inexistente para este voo.' AS Resultado;
ELSEIF v_status != 'DISPONIVEL' THEN
ROLLBACK;
SELECT 'Operação negada: O assento já está reservado ou bloqueado.' AS Resultado;
ELSE
UPDATE assentos SET status = 'RESERVADO' WHERE id = v_assento_id;
INSERT INTO reservas (passageiro_id, assento_id)
VALUES (p_passageiro_id, v_assento_id);
COMMIT;
SELECT 'Sucesso! Reserva confirmada.' AS Resultado;
END IF;
END $$

DELIMITER ;

```

Evidências da Execução

Teste 1 — Reserva Válida (Passageiro 1)

Comando: CALL ReservarAssento(1, 1, '10A');

Retorno: Sucesso! Reserva confirmada.

Teste 2 — Disputa pelo Mesmo Assento (Passageiro 2)

Comando: CALL ReservarAssento(2, 1, '10A');

Retorno: Operação negada: O assento já está reservado ou bloqueado. A lógica interceptou a mudança baseada no bloqueio serializado da primeira chamada.